

信息学中概率类题目的分析

上海市上海中学 swj05652

关键词： 信息学 概率 动态规划

这篇文章中的“概率类题目”指的是题目所求为某一事件的发生几率或是某一个量的平均意义下的值。通常要求输出一个小数或是一个最简分数。这类题目通常会涉及到递推或是动态规划的思想。一般来说要抓住这一点去进行思考。下面我们来看几道典型题目。

[终极情报网](#)

[神奇口袋](#)

[百事世界杯之旅](#)

[单人纸牌](#)

[三国围棋对抗赛](#)

[并行期望值](#)

[迷宫统计](#)

[神经衰弱](#)

[聪聪和可可](#)

[巧克力](#)

[总结](#)

终极情报网

agent.pas (exe)

【题目叙述】

在最后的诺曼底登陆战开始之前，盟军与德军的情报部门围绕着最终的登陆地点展开了一场规模空前的情报战。

这场情报战中，盟军的战术是利用那些潜伏在敌军内部的双重间谍，将假的登陆消息发布给敌军的情报机关的负责人。那些已经潜入敌后的间谍们都是盟军情报部的精英，忠实可靠；但是如何选择合适的人选，以及最佳的消息传递方法，才能保证假消息能够尽快而且安全准确地传递到德军指挥官们的耳朵里，成了困扰盟军情报部长的最大问题。他需要你的帮助。

以下是情报部长提供的作战资料：

在敌后一共潜伏着我方最优秀的 N 名间谍，分别用数字 $1, 2, \dots, N$ 编号。在给定的作战时间内，任意两人之间至多只进行一次点对点的双人联系。

我将给你一份表格，表格中将提供任意两位间谍 i 和 j 之间进行联系的安全程度，用一个 $[0, 1]$ 的实数 S_{ij} 表示；以及他们这次联系时，能够互相传递的消息的最大数目，用一个正整数表示 M_{ij} (如果在表格中没有被提及，那么间谍 i 和 j 之间不进行直接联系)。

假消息从盟军总部传递到每个间谍手里的渠道也不是绝对安全，我们用 $[0, 1]$ 的实数 AS_j 表示总部与间谍 j 之间进行联系的安全程度， AM_j 则表示总部和间谍 j 之间进行联系时传递的消息的最大数目。对于不和总部直接联系的间谍，他的 $AM_j=0$ (而表格中给出的他的 AS_j 是没有意义的)。

当然，假消息从间谍手中交到敌军的情报部官员的办公桌上的过程是绝对安全的，也即是说，间谍与敌军情报部门之间要么不进行直接联系，要么其联系的安全程度是 1 (即完全可靠)。

现在情报部打算把 K 条假消息“透露”到德军那里。消息先由总部一次性发给 N 名间谍中的一些人，再通过他们之间的情报网传播，最终由这 N 名间谍中的某些将情报送到德军手中。

对于一条消息，只有安全的通过了所有的中转过程到达敌军情报部，这个传递消息的过程才算是安全的；因此根据乘法原理，它的安全程度 P 就是从总部出发，经多次传递直到到达德军那里，每一次传递该消息的安全程度的乘积。

而对于整个计划而言，只有当 N 条消息都安全的通过情报网到达德军手中，没有一条引起怀疑时，才算是成功的。所以计划的可靠程度是所有消息的安全程度的乘积。

显然，计划的可靠性取决于这些消息在情报网中的传递方法。

我需要一个方案，确定消息应该从哪些人手中传递到哪些人手中，使得最终

计划的可靠性最大。

你可以利用计算机，来求得这个最可靠的消息传递方案。

【输入文件】

输入文件 `agent.in` 包含了盟军的作战资料表格。

第一行包括两个整数 N 和 K ，分别是间谍的总人数和计划包含的消息总数。

第二行包括 $2N$ 个数，前 N 个数是实数 $AS_1, AS_2, \dots, AS_N$ （范围在 $[0, 1]$ 以内）；后 N 个数是整数 $AM_1, AM_1, \dots, AM_N$ 。

第三行包含了 N 个整数，其中第 i ($i = 1, 2, \dots, N$) 个整数如果为 0 表示间谍 i 与德军情报部不进行联系，如果为 1 则表示间谍与德军情报部进行联系。

第四行开始，每行包括 4 个数，依次分别是：代表间谍编号的正整数 i 和 j ，间谍 i 和 j 联系的安全性参数 S_{ij} ($[0, 1]$ 范围内的实数)，以及 i, j 之间传递的最大消息数 M_{ij} （每一行的 i 均小于 j ）。

最后的一行包含两个整数 -1 -1，表示输入数据的结束。

$0 < N < 300$ ； $0 < K < 300$ 。

【输出文件】

输出文件 `agent.out` 中只有一行。这一行中包含一个实数 P ，给出的是整个计划的可靠程度 P ，保留 5 位有效数字（四舍五入）。

如果情报网根本不能将 K 条消息传到德军手中，那么计划的可靠性为 0。

（你可以假定，如果计划存在，那么它的可靠性大于 $1e-12$ ）

【输入输出样例】

Agent.in										
6	13									
0.9	0.7	0.8	0	0	0	2	6	8	0	0
0	0	0	1	0	1					
1	4	0.5	2							
2	3	0.9	5							
2	5	0.8	2							
2	6	0.8	7							
3	5	0.8	2							
5	6	0.8	4							
-1	-1									

Agent.out
0.00021184

【算法分析】

其实就是网络流最小费用算法。

首先要把所有概率值去自然对数，把乘法运算转变为加法运算，这样既提高运算速度、精度，也更容易看清题目的本质——就是一个网络，给出流量上界、单位流量费用，求指定流量的最小费用。

最大流最小费用是一个必须掌握的基本算法，由于最大流最小费用保证每一布增广出来的流都是最优，这里只要套用就行了。每次做增广路径，看流量是否达到或超过了 K ，超过的话就在调整过程中不要“满负荷”。

(From CTSC 2001 Day1)

神奇口袋

【题目叙述】

Pòlya 获得了一个奇妙的口袋，上面写着人类难以理解的符号。Pòlya 看得入了迷，冥思苦想，发现了一个神奇的模型（被后人称为 Pòlya 模型）。为了生动地讲授这个神奇的模型，他带着学生们做了一个虚拟游戏：

游戏开始时，袋中装入 a_1 个颜色为 1 的球， a_2 个颜色为 2 的球，...， a_t

$$a_i \in \mathbb{Z}^+ (1 \leq i \leq t)。$$

个颜色为 t 的球，其中

游戏开始后，每次严格进行如下的操作：

从袋中随机的抽出一个小球（袋中所有小球被抽中的概率相等），Pòlya 独自观察这个小球的颜色后将其放回，然后再把 d 个与其颜色相同的小球放到口袋中。

设 c_i 表示第 i 次抽出的小球的颜色 ($1 \leq c_i \leq t$)，一个游戏过程将会产生一个颜色序列 $(c_1, c_2, \dots, c_n, \dots)$ 。

Pòlya 把游戏开始时 t 种颜色的小球每一种的个数 $a_1, a_2, \dots, a_t$ 告诉了所有学生。然后他问学生：一次游戏过程产生的颜色序列满足下列条件的概率有多大？

$$c_{x_1} = y_1, c_{x_2} = y_2, \dots, c_{x_i} = y_i, \dots, c_{x_n} = y_n$$

其中 $0 < x_1 < x_2 < \dots < x_n$ ， $1 \leq y_i \leq t$ 。换句话说，已知 $(t, n, d, a_1, a_2, \dots, a_t, x_1, y_1, x_2, y_2, \dots, x_n, y_n)$ ，你要回答有多大的可能性会发生下面的事件：“对所有 $k, 1 \leq k \leq n$ ，第 x_k 次抽出的球的颜色为 y_k ”。

【输入文件】

第一行有三个正整数 t, n, d ；第二行有 t 个正整数 $a_1, a_2, \dots, a_t$ ，表示游戏开始时口袋里 t 种颜色的球，每种球的个数。

以下 n 行，每行有两个正整数 x_i, y_i ，表示第 x_i 次抽出颜色为的 y_i 球。

【输出文件】

要求用分数形式输出（显然此概率为有理数）。输出文件包含一行，格式为：分子/分母。同时要求输出最简形式（分子分母互质）。特别的，概率为 0 应输出 0/1，概率为 1 应输出 1/1。

【输入输出样例】

样例 1 的输入	样例 1 的输出
2 3 1 1 1 1 1 2 2 3 1	1/12

样例 2 的输入	样例 2 输出
3 1 2 1 1 1 5 1	1/3

【样例1 说明】

初始时，两种颜色球数分别为(1, 1)，取出色号为 1 的球的概率为 1/2；第二次取球之前，两种颜色球数分别为(2, 1)，取出色号为 2 的球的概率为 1/3；第三次取球之前，两种颜色球数分别为(2, 2)，取出色号为 1 的球的概率为 1/2，所以三次取球的总概率为 1/12。

【数据规模和约定】

$1 \leq t, n \leq 1000, 1 \leq a_k, d \leq 10, 1 \leq x_1 < x_2 < \dots < x_n \leq 10000, 1 \leq y_k \leq t$

【评分方法】

本题没有部分分，你的程序的输出只有和我们的答案完全一致才能获得满分，否则不得分。

【算法分析】

这是这次 NOI 最简单的一道题，考察的只是很简单的数学知识+高精度乘法。

简单一些的描述就是每次随机选择一种颜色的球，将数量+d。整个大框架就是乘法原理，对于每次取球，如果没有在 x_i 中，那么这次操作称为“自由取球”，否则将“颜色 y_i 的球的数量 a_i / 此时球的总数”作为乘数乘到结果中，并将颜色 y_i 的球的数量+d，称作“定向取球”

可以证明，自由取球对任意球被取到的几率 P_i 没有影响。证明很简单，就不说了。

这样就可以忽略掉所有的自由取球操作，即每一步都是定向取球。剩下的就只是高精度乘法了

对于最后分数化简的问题，因为分子分母最多只有 20000，所以只要建立一个 20000 以内的素数表。连高精度除法都不需要，只要先将分子分母都化简掉再高精度乘起来输出就行了。

(From NOI 2006 Day2)

百事世界杯之旅

(PEPSI)

“……在 2002 年 6 月之前购买的百事任何饮料的瓶盖上都会有一个百事球星的名字。只要凑齐所有百事球星的名字，就可以参加百事世界杯之旅的抽奖活动，获取球星背包、随身听，更可以赴日韩观看世界杯。还不赶快行动！……”

你关上电视，心想：假设有 n 个不同的球星名字，每个名字出现的概率相同，平均需要买几瓶饮料才能凑齐所有的名字呢？

输入输出要求

输入一个数字 n ， $2 \leq n \leq 33$ ，表示不同球星名字的个数。

输出凑齐所有的名字平均需要购买的饮料瓶数。如果是一个整数，则直接输出，否则用下面样例中的格式分别输出整数部分和小数部分。分数必须是不可约的。

【输入输出样例】

样例一

输入	输出
2	3

样例二

输入	输出
5	5 11— 12

样例三

输入	输出
17	340463 58—— 720720

(From SHTSC 2002 Day1)

【算法分析】

这道题目主要考察的是数学的推导能力。假设 $f[n, k]$ 为一共有 n 个球星，现在还剩 k 个未收集到，还需购买饮料的平均次数。则有：

$$f(n, k) = \frac{(n-k)f(n, k)}{n} + \frac{kf(n, k-1)}{n} + 1$$

经移项整理，可得：

$$f(n, k) = f(n, k-1) + \frac{n}{k}$$

我们所要求的实际上应该是 $f(n, n)$ ，根据 $f(n, k)$ 的递推式，可得

$$f(n, n) = n \sum_{k=1}^n \frac{1}{k} = nH(n)$$

其中， $H(n)$ 表示 Harmonic Number。

单人纸牌

【题目叙述】

Double Patience是一种单人纸牌游戏，共36张牌分成9叠，每叠4张牌面向上。

每次，游戏者可以从某两个不同的牌堆最顶上取出两张牌面相同的牌（如黑桃10和梅花10）并且一起拿走。如果最后所有纸牌都被取走，则游戏者就赢了，否则游戏者就输了。

George很热衷于玩这个游戏，但是一旦有时有多种选择的方法，George就不知道取哪一种好了，George会从中随机地选择一种走，例如：顶上的9张牌为KS, KH, KD, 9H, 8S, 8D, 7C, 7D, 6H，显然有5种取法：(KS, KH), (KS, KD), (KH, KD), (8S, 8D), (7C, 7D)，当然George取到每一种取法的概率都是1/5。

有一次，George的朋友Andrew告诉他，这样做是很愚蠢的，不过George不相信，他认为如此玩最后成功的概率是非常大的。请写一个程序帮助George证明他的结论：计算按照他的策略，最后胜利的概率。

【输入文件】

9行每行4组用空格分开的字串，每个字串两个字符，分别表示牌面和花色，按照从堆底到堆顶的顺序给出。

【输出文件】

一行，最后胜利的概率，精确到小数点后6位。

【输入输出样例】

Double.in	Double.out
AS 9S 6C KS	0.589314
JC QH AC KH	
7S QD JD KD	
QS TS JS 9H	
6D TD AD 8S	
QC TH KC 8D	
8C 9D TC 7C	
9C 7H JH 7D	
8H 6S AH 6H	

【算法分析】

设计状态： $f(a, b, c, d)$ 表示四堆牌各剩下 a, b, c, d 张时胜利的概率。计算 $f(a, b, c, d)$ 时，先看牌顶的取法有几对，如果无法取则 $f(a, b, c, d)=0$ 。假设 a 和 b 的最上面一张可以拿走，那么就将 $f(a-1, b-1, c, d)$ 作为 $f(a, b, c, d)$ 的一个前继节点。 $f(a, b, c, d)$ 所有前继节点的平均值就是 $f(a, b, c, d)$ 的值。

初始节点为 $f(0, 0, 0, 0)=1$ 。具体实现时可以用记忆化递归。鉴于题目的数据规模，很多优化不必作了（比方说， $a+b+c+d$ 只有为偶数才有意义）

三国围棋对抗赛

【题目叙述】

中国邀请韩国，日本围棋队来参加三国围棋对抗赛，韩国，日本应邀各派了 5 位超一流高手来参赛，中国围棋队希望能赢得这场比赛，但是这 10 位高手实力不俗。不过中国队作为东道主，可以在对方选手安排好出场顺序后再决定队员的组成以及出场顺序，以得到最大的获胜概率。

比赛规则如下：先抽签决定第 1 轮轮空的队，由不轮空的 2 支队的 1 号队员进行比赛，失利的队员被淘汰，以后每次由前一轮获胜的队员与前一轮轮空的队剩下的队员中序号最小的队员进行比赛，直到只剩下一个国家的队员为止，这个国家就获得了比赛的胜利。

【输入文件】

第 1 行为一个数 n ($5 \leq n \leq 15$)，中国队的候选人数。

接下来是 n 行，每行有 10 个数

第 $I+1$ 行的 10 个数依次表示第 I 位中国选手对韩国 1~5 号，日本 1~5 号的胜率，胜率 k ($0 \leq k \leq 1$)。

再接下来是 5 行，每行有 5 个数

第 $I+n+1$ 行的第 j 个数表示韩国 I 号选手对日本 j 号选手的胜率。

【输出文件】

仅 1 行，为中国队的最大的获胜概率，保留 6 位小数。

【输入输出样例】

15

```
0.332 0.683 0.622 0.072 0.312 0.184 0.658 0.358 0.849 0.211
0.004 0.466 0.462 0.024 0.051 0.406 0.354 0.637 0.390 0.795
0.285 0.559 0.359 0.760 0.686 0.554 0.190 0.354 0.216 0.673
0.169 0.689 0.681 0.648 0.135 0.321 0.192 0.765 0.110 0.516
0.788 0.472 0.238 0.539 0.306 0.564 0.347 0.702 0.814 0.630
0.003 0.803 0.349 0.897 0.871 0.434 0.669 0.351 0.713 0.256
0.257 0.085 0.819 0.727 0.345 0.877 0.643 0.065 0.242 0.452
0.183 0.688 0.149 0.269 0.095 0.805 0.030 0.201 0.146 0.682
0.816 0.229 0.846 0.332 0.586 0.330 0.375 0.787 0.816 0.345
0.312 0.026 0.407 0.082 0.134 0.757 0.164 0.771 0.286 0.184
0.215 0.396 0.696 0.246 0.318 0.699 0.570 0.067 0.801 0.019
0.271 0.276 0.420 0.711 0.219 0.066 0.767 0.521 0.035 0.692
0.820 0.128 0.248 0.633 0.720 0.281 0.612 0.812 0.670 0.724
0.356 0.819 0.289 0.012 0.786 0.616 0.448 0.043 0.164 0.661
0.431 0.802 0.524 0.798 0.078 0.862 0.139 0.889 0.602 0.637
0.235 0.119 0.713 0.201 0.611
0.069 0.268 0.589 0.073 0.912
0.047 0.347 0.087 0.721 0.170
0.408 0.769 0.337 0.888 0.288
```

0.902 0.860 0.479 0.299 0.329

输出：

0.624032

【算法分析】

首先枚举中国出场的 5 个人（题意应该是这样的，每个国家都只出 5 人。从测试数据中可以看出）。这一步的复杂度 $15 \times 14 \times 13 \times 12 \times 11 = 3.6 \times 10^5$ 。

设计状态 $f(i,j,k,o)$ ： i,j,k 表示三国各剩多少人（1..5）， o 表示当轮轮空的国家（1..3）。转移方程和初始状态都不难想，这里不再赘述（就是写起代码来很长）。复杂度约为 $5 \times 5 \times 5 \times 3 \times 3 = 1225$ 。

题目给出的时限是 20s，实际上测试数据没有 $n > 10$ 的。在 Pentium 4 1.70GHz CPU 上 $n=15$ 大约 17s。 $n=10$ 在 5s 之内。考虑到当时 cpu 没有那么快，这个程序应当可以勉强过关。这个算法仍然有待改进。（比如搜索中如何剪枝。每一次搜索都要做一下递推，能否利用上次的结果。空间消耗很小，怎么拿空间换时间。这三大问题其实是一体的，一旦解决，时间效率将有极大的提高。）

(From SHTSC 2001)

并行期望值

考虑一个可以并行执行两个高级语言程序的机器。每个高级语言程序由若干条指令构成均为赋值指令，它的形式是：

$\langle \text{变量} \rangle := \langle \text{运算数 1} \rangle \text{ op } \langle \text{运算数 2} \rangle$

其中，变量名由不超过 20 个字母组成。运算数是一个变量名或者小于 100 的正整数。op 是运算符，它可以是加号“+”或者减号“-”。

执行之前，高级语言程序首先被翻译成机器语言。一条语句 $X := Y + Z$ 被翻译成：

Mov R1, Y

Mov R2, Z

Add R1, R2

Mov X, R1

Mov 指令把第二个操作数送到第一个中去，Add(Sub)操作进行加法（减法），并把结果保存在第一个操作数中。注意 Y 和 Z 代表变量或者整数常量。 $X := Y - Z$ 的机器语言代码类似，只是 Add 指令被 Sub 指令代替了。

处理器接受了两个机器语言程序后，每次随机选一个程序，执行它的下一条指令。如果某个程序执行完毕，处理器会连续把另一个程序执行完。两个程序共享所有变量（一开始的时候各个变量的值均为 0），但是拥有两个独立的寄存器 R1 和 R2。

由于执行顺序不确定，最后各个变量的值也是不确定的。不过，可以计算出每个变量在所有情况下最终值的平均数（即并行期望值）。现在给你两个高级语言程序，请编程算出所有变量的并行期望值。每个高级语言程序最多有 25 条指令，两个程序最多共使用 10 个变量。

【分析】

首先，读入数据并将高级语言程序转换成机器语言程序并保存起来，每次将一条高级语言的赋值指令：

$\langle \text{变量} \rangle := \langle \text{运算数 1} \rangle \text{ op } \langle \text{运算数 2} \rangle$

转换成形式一致的四条机器语言的赋值指令：

$\langle \text{Rx1} \rangle := \langle \text{运算数 1} \rangle + \langle 0 \rangle$

$\langle \text{Rx2} \rangle := \langle \text{运算数 2} \rangle + \langle 0 \rangle$

$\langle \text{Rx1} \rangle := \langle \text{Rx1} \rangle \text{ op } \langle \text{Rx2} \rangle$

$\langle \text{变量} \rangle := \langle \text{Rx1} \rangle + \langle 0 \rangle$

如果是转换程序 1 中的高级指令，那么 $x = a$ ；如果是转换程序 2 中的高级指令，那么 $x = b$ 。这使得两程序分别拥有了独立的寄存器 Ra1、Ra2 和 Rb1、Rb2，同时又将寄存器一般化为普通变量，方便了以后的处理。接下来的讨论均是针对机器语言的。

指令 (w, i) 表示程序 w 的第 i 条指令。变量 $d[x, y, k]$ 表示程序 1 执行完毕 x 条指令，程序 2 执行完毕 y 条指令后（即状态 $\langle x, y \rangle$ ）变量 k 的并行期望值。序列 $\{p_1, p_2, p_3, \dots, p_{x+y}\}$ 是一条达到状态 $\langle x, y \rangle$ 的路径的具体描述，其中 $p_i = 0$ 表示第 i 次机器选择程序 1 中的指令

执行, 否则相应的, $p_i = 1$ 表示第 i 次机器选择程序 2 中的指令执行。按照序列 $\{p_1, p_2, p_3, \dots, p_{x+y}\}$ 并行执行两个程序所得变量 k 的值称为状态 $\langle x, y \rangle$ 下序列 p 的 k 值。很明显,

$$d[x, y, k] = \frac{\text{状态} \langle x, y \rangle \text{下所有不同序列的} k \text{值和}}{\text{状态} \langle x, y \rangle \text{下不同序列的总数}}$$

根据排列组合初步知识, 状态 $\langle x, y \rangle$ 下不同序列的总数 $= C_{x+y}^x$ 。

考虑 $x, y > 0$ 的一般情况, 状态 $\langle x, y \rangle$ 的 k 值和 $=$ 状态 $\langle x-1, y \rangle$ 执行 (1, x) 后的 k 值和 (记为 k_{sum1}) $+$ 状态 $\langle x, y-1 \rangle$ 执行 (2, y) 后的 k 值和 (记为 k_{sum2})。怎么求状态 $\langle x-1, y \rangle$ 执行 (1, x) 后的 k 值和呢? 分情况讨论:

(1) 指令 (1, x) 中被赋值的变量不是 k , 那么 $k_{\text{sum1}} =$ 状态 $\langle x-1, y \rangle$ 的 k 值和。

(2) 指令 (1, x) 形如 $k := \langle 1_a \rangle \text{ op } \langle 1_b \rangle$

为方便叙述, 这里把常数看作拥有固定值的变量。假设状态 $\langle x-1, y \rangle$ 下序列 P_i 的 k 值为 K_i , 1_a 值为 A_i , 1_b 值为 B_i 。易知, 序列 $\{P_i, 0\}$ 的 k 值为 $A_i \text{ op } B_i$ 。现在考虑状态 $\langle x-1, y \rangle$ 下的所有序列 $P_1, P_2, P_3, \dots, P_{\text{sum}}$, 它们执行指令 (1, x) 后的 k 值分别为 $A_1 \text{ op } B_1, A_2 \text{ op } B_2, A_3 \text{ op } B_3, \dots, A_{\text{sum}} \text{ op } B_{\text{sum}}$ 。

Sum 是个巨大的数, 大到时间和空间上都不允许记录每个序列 P_i 相应的 A_i 和 B_i 值。但幸运的是, 我们无需知道 A_i, B_i 的具体值。为什么呢? 由于加法交换律及合并律的存在,

$$K_{\text{sum1}} = \sum_{i=1}^{\text{sum}} K_i = \sum_{i=1}^{\text{sum}} (A_i \text{ op } B_i) = \left(\sum_{i=1}^{\text{sum}} A_i \right) \text{ op } \left(\sum_{i=1}^{\text{sum}} B_i \right)$$

也就是说, 只要知道状态 $\langle x-1, y \rangle$ 的 A 值和 B 值和, K_{sum1} 就通过简单的 op 运算得出了。发现了问题的最优子结构, 剩下的工作就很简单了。

$$\begin{aligned} \text{令 } K_1 &= \begin{cases} d[x-1, y, k] & \text{(情况(1))} \\ d[x-1, y, 1_a] \text{ op } d[x-1, y, 1_b] & \text{(情况(2))} \end{cases} \\ K_2 &= \begin{cases} d[x, y-1, k] & \text{(情况(1))} \\ d[x, y-1, 2_a] \text{ op } d[x, y-1, 2_b] & \text{(情况(2))} \end{cases} \end{aligned}$$

然后整理一下得: 状态 $\langle x, y \rangle$ 的 k 值和 $= K_{\text{sum1}} + K_{\text{sum2}} = K_1 \times C_{x+y-1}^{x-1} + K_2 \times C_{x+y-1}^{y-1}$ 。状态转移方程为:

$$d[x, y, k] = \frac{K_1 \times C_{x+y-1}^{x-1} + K_2 \times C_{x+y-1}^{y-1}}{C_{x+y}^x} = \frac{K_1 \times \frac{(x+y-1)!}{(x-1)!y!} + K_2 \times \frac{(x+y-1)!}{x!(y-1)!}}{\frac{(x+y)!}{x!y!}} = \frac{K_1 \times x + K_2 \times y}{x+y}$$

时间复杂度仅为 $O(l_1 l_2 n)$, 其中 l_1, l_2 分别为两个程序的长度, n 为变量个数。本题的巧妙之处是发现了最优子结构, 这也是在线性模型中设计动态规划的主要障碍。

(From 《算法艺术与信息学竞赛》 P125)

迷宫统计

Jimmy 自己办了一个游园活动，其中一个项目是让游客去走一个随机生成的 m 行 $n()$ 列的迷宫，迷宫里有空地，也有障碍物（每个障碍物恰好占一个方格）。游客总是从左上角走到右下角，每次可以往东南西北四个方向之一行走。迷宫的生成方式很简单：每一个格子都有一个独立的概率 p ，表示该格子为障碍物的概率。如果程序生成了一个误解的迷宫，他会重新生成一个。你的任务是计算每个格子成为一个有解迷宫中的障碍物的概率。

【分析】

本题是一道难题，比赛时 380 只队伍中仅有两人做出此题。此题难就难在即使在 $m \leq 5, n \leq 6$ 的时候，有解迷宫也是很多的。 $m=n=5$ 时有解迷宫有 1 225 194 个； $m=5, n=6$ 时高达 30 754 544 个。如果把所有有解迷宫都列举出来再计算概率，需要花费约 10 分钟的时间。我们的思路是：不列举所有有解迷宫，而是把这些迷宫分成若干个不相交的集合，在每个集合中分别计算概率。这样，得到了算法一。

算法一：根据特征通路的存在性分类

让 $\text{maze}[x][y] = 0$ 表示 (x,y) 必须有障碍， $\text{maze}[x][y] > 1$ 表示 (x,y) 必须是空地， $\text{maze}[x][y] = 1$ 表示 (x,y) 可以是任意形式，则一个 maze 数组对应了一个迷宫（不一定是有解的）集合，把 maze 叫做该迷宫集合的模板。初始的时候，所有合法迷宫组成的集合的模板满足：起点和终点元素为 2（必须是空地），而所有其他元素均为 1（任意形式）。

对于任意一个模板 M ，随便在这个模板中找一条左上角到右下角的通路 P （通路中的元素满足 $\text{maze}[x][y] > 0$ ），则可以把这个模板所对应的集合划分成两个不相交的集合：用 A 来表示所有存在通路 P 的迷宫， B 来表示所有不存在通路 P 的迷宫，则把 P 中元素的 maze 值全部加 1（表示这些格子必须是空地），得到的模板即为 A 的模板。 B 的模板比较复杂，还需要进一步把它分类。

既然不存在通路 P ，显然 P 上至少应该有一个位置必须有障碍。用 $B(i)$ 代表通路 P 上的最后一个障碍位置为 i 的迷宫所对应的模板，则 $B(i)$ 应该是把位置 i 处设置为 0（障碍），位置 i 以后的设置为大于 1（空地，因为 i 是最后一个障碍），而位置 i 以前的设置为 1（无论前面的格子是什么，通路 P 一定不存在）。设 P 的长度为 k ，则把模板 M 分解为了 $k+1$ 个不相交的模板： $A, B(1), B(2), \dots, B(k)$ 。这样递归下去就可以了。模板的递归边界是所有一定含某通路 P 的 A 类集合，只需要处理每个这样的模板所对应的迷宫就可以了。

给出一个模板 M ，设满足 $\text{maze}[x][y]=0$ 的格子集合为 S_1 ，满足 $\text{maze}[x][y] > 1$ 的格子集合为 S_2 ，设满足 $\text{maze}[x][y]=1$ 的格子集合为 S_3 ，那么模板 M 所对应的所有迷宫（这些迷宫都有解，因为包含一条通路）的概率和为：

$$\text{Prob}(M) = (S_1 \text{ 每个元素为障碍的概率乘积}) \times (S_2 \text{ 每个元素为空地的概率乘积})$$

如图 1-79 所示，左边的矩阵表示各格子为障碍的概率，右边表示模板。

0.0	0.1	0.2	空地(2)	空地(2)	空地(2)
0.3	0.4	0.5	未知(1)	未知(1)	空地(2)
0.6	0.7	0.8	障碍(0)	障碍(0)	空地(2)

图 1-79 模板和它的概率矩阵

则 S_1 每个元素为障碍的概率乘积为 $0.6 \times 0.7 = 0.42$ ， S_2 每个元素为空地的概率乘积为 $1.0 \times 0.9 \times 0.8 \times 0.5 \times 0.2 = 0.072$ ，因此这个模板所对应的有解迷宫总概率为 0.072。

对于每个格子 (x, y) ，还需要统计在模板 M 中，它为障碍的迷宫总概率。显然当 $\text{maze}[x,y]$

= 0 时, 这个概率就是 $\text{Prob}(M)$; 如果 $\text{maze}[x,y]>1$, 这个概率为 0; 如果 $\text{maze}[x,y]=1$, 则这个概率为 $\text{Prob}(M) \times ((x,y) \text{ 为障碍物的概率})$

这样, 把所有 A 类模板对应的迷宫总概率加起来, 得到所有有解迷宫的总概率 ProbAll , 另外还统计了所有 A 类模板中, 每个格子 (x,y) 为障碍时的概率总和 $\text{ProbBlock}(x,y)$, 由条件概率公式, 则所有答案 $\text{ans}(x,y) = \text{ProbBlock}(x,y) / \text{ProbAll}$ 。

程序的运行时间已经缩短到了 3 秒, 但是仍然认为集合划分得太细致, 导致计算量仍很大。有没有其他方法呢? 尝试上题的思路一列一列地进行状态压缩。

算法二: 基于状态压缩的动态规划

既然是一列一列递推, 需要知道当前列 (或者多列) 需要记录哪些信息, 即哪些信息对最后的答案有用。上题中, 需要保存最后两行, 但是本题保存一列、两列或者三列都可能不够。如果需要保存完, 那么状态就太多, 怎么办呢? 一种想法是只记录当前列的每个格子是否能和起点连通。但是这样做是不对的, 因此即使当前某个格子和起点不连通, 以后也是可能连通的。这样做在状态转移的时候遇到了困难。正确的做法是另外记录当前列每两个格子是否连通, 即记录在当前列 y 中, 每个格子 (x,y) 的 $\text{last}[x]$ 值, 其中 $\text{last}[x]=0$ 表示它和起点连通, 否则它表示在 (x,y) 正上方与 (x,y) 连通的格子中的最小行编号 (如果没有这样的格子, 则 $\text{last}[x] = x$)。这样, 用 $(i, \text{last}[])$ 表示一个状态, 即前 i 列, 第 i 列的 last 数组为 $\text{last}[]$ 的所有迷宫集合, 并用 $d[i, \text{last}]$ 表示这些迷宫的总概率。为了统计和各个格子相关的概率, 还需要增加一维 b , 用 $d[i, \text{last}, b]$ 表示, 前 i 列, 第 i 列的 last 数组为 $\text{last}[]$, 其中第 b 个格子为障碍的迷宫总概率 ($0 \leq b \leq mn, j=0$ 表示总概率)。

这样, 每次进行状态转移时, 枚举当前列的所有 $(m+1)!(mn+1)$ 种状态和 2^m 种决策 (下一列的障碍情况), 状态转移的时候需要做 BFS, 但是由于只需要用上一列的 last 值和当前列, 因此状态转移的复杂度为 2^{m+1} , 而且当 i 和 last 一定时, 对于不同的 b 计算 $d[i, \text{last}, b]$ 只需要 BFS 一次, 因此算法框架为:

```
for i := 1 to n do
  for k := 1 to (m+1)! do
    begin
      计算出第 k 个可能的 last[] 数组;
      for p:=1 to 2m do
        begin
          计算出第 p 个可能的当前列
          花 2m+1 的时间做一次 BFS
          for b := 0 to m*n do
            递推 d[i, last, b]
          end;
        end;
      end;
    end;
```

总的计算量约为 $n(m+1)!2^m(mn+1+2^{m+1}) = O(mn^2 2^m m!)$ 。显然和 m 有关的数比 n 大得多, 因此总认为 $m \leq n$, 否则把 m, n 交换, 并把矩阵翻转。和上题一样, 可以用滚动数组, 这样空间需求是 $O((m+1)!mn)$, 这样的程序只需要不到 0.2 秒就得到了正确的结果。

(From 《算法艺术与信息学竞赛》 P139)

神经衰弱 (shinkei)

【题目叙述】

有个游戏相信大家都不陌生，拿一副 54 张的扑克牌，面朝下铺开在桌子上。一个人每次翻开两张牌，如果两张牌点数一样，则把这两张牌同时拿走，否则的话，就还是把两张牌翻回去。就这样继续，直到所有牌全部被拿完。看自己可以在多少轮内把牌拿完。在日本，一般管这个游戏叫神经衰弱。

我们把这个问题扩展到更一般的情形。假设有 n 种不同的牌，每一种牌有完全相同的 $2m$ 张。假设你是一个记忆力非常好的人，只要看过一次的牌就绝对不会忘记。并且，你总是按照这样的步骤进行游戏：

- 如果存在至少一对已知的牌，则下一步一定是将这两张牌翻开并拿走，如果这样的牌不止一对，则从其中任意选择一对。
- 如果不存在已知的相同的牌，则首先从未知的牌中任意翻开一张。
 - 如果这张牌与已知的某张牌相同，则将那张牌翻开并拿走这一对牌。
 - 如果不存在已知的牌与这张牌相同，则从未知的牌中再任意翻开一张。如果两张牌恰巧相同，则拿走这一对牌。

这样，你总是重复以上的步骤直到所有的牌都被拿完。当然，由于运气的好坏不同，每次拿完所有的牌所需要的论述的平均值（数学期望）究竟是多少？这里，定义每翻开两张牌（无论相同与否）为一轮。

【输入文件】

输入数据仅有一行，为两个整数 n 和 m ($1 \leq n \leq 16$, $1 \leq m \leq 4$)，两个数字用一个空格分开。

【输出文件】

输出在该情况下拿完所有牌需要的轮数的数学期望，结果四舍五入到小数点后 6 位小数。

【输入输出样例】

shinkei.in
2 1

sheinkei.out
2.666667

【分析】

我们同样来关注状态的设计。从 n 入手是比较容易想到的。也就是 n 维状态，每一维表示还剩几对，整个状态的值表示还需要几回合完成游戏。这样空间需求为 $4^{16}=4096\text{MB}$ ，是不现实的。

m 的数据范围提示我们应当从此入手，设计一个 m 维的状态。我们又会注意到每一种牌至多已知一张牌的位置（如果知道两张就可以立即翻掉了），那么我们设计 $f(a_1, a_2, a_3, a_4, b_1, b_2, b_3, b_4)$ 。其中 a_i 表示剩下 i 对的牌有几种。 B_i 表示 a_i 这些牌中有多少种已知单牌。

状态转移方程有点复杂，需要考虑到多种情况：

- (1) 翻开一张，与已知的相配。代价为 1，加上后继节点 $f(a_1-1, a_2, a_3, a_4, b_1-1, b_2, b_3, b_4)$ （或是其它 3 种），乘上概率
- (2) 乱翻两张，居然正好凑巧摸到了之前不知道单牌的一对，代价为 1，加上

后继节点 $f(a1-1, a2, a3, a4, b1, b2, b3, b4)$ (或是其它 3 种), 乘上概率.

(3) 翻开一张, 增加一张单牌, 再翻一张, 可以与之前某张配对. 后继节点为 $f(a1-1, a2, a3, a4, b1-1, b2, b3+1, b4)$ 之类的, 代价为 2.

(4) 翻开两张, 增加两张单牌, 代价 1 转移到 $f(a1, a2, a3, a4, b1, b2+1, b3, b4+1)$ (只要 4 个 b 中间加两个 1 或是加一个 2)

这仅仅是四个大类, 每一类分别对应一段代码, 每一类中的转移都有若干种, 要仔细写好概率的计算式。

也许有同学会怀疑: 这种状态设计空间不仍然是 $16^8=4096MB$ 吗? 其实 $a1+a2+a3+a4 \leq 16$, 所以把这四个参量压缩成一个。大约需要 1~4067 的范围。 $B1, b2, b3, b4$ 也如法炮制, 空间需求达到了 16MB, 时间复杂度约为 $1E8$ 。做好四维压缩的映射表, 仔细考虑好递推顺序。

(From SHTSC 2006)

聪聪和可可

(cchkk)

【题目叙述】

在一个魔法森林里，住着一只聪明的小猫聪聪和一只可爱的小老鼠可可。虽然灰姑娘非常喜欢她们俩，但是，聪聪终究是一只猫，而可可终究是一只老鼠，同样不变的是，聪聪成天想着要吃掉可可。

一天，聪聪意外得到了一台非常有用的机器，据说是叫 GPS，对可可能准确的定位。有了这台机器，聪聪要吃可可就易如反掌了。于是，聪聪准备马上出发，去找可可。而可怜的可可还不知道大难即将临头，仍在森林里无忧无虑的玩耍。小兔子乖乖听到这件事，马上向灰姑娘报告。灰姑娘决定尽快阻止聪聪，拯救可可，可她不知道还有没有足够的时间。

整个森林可以认为是一个无向图，图中有 N 个美丽的景点，景点从 1 至 N 编号。小动物们都只在景点休息、玩耍。在景点之间有一些路连接。

当聪聪得到 GPS 时，可可正在景点 $M(M \leq N)$ 处。以后的每个时间单位，可可都会选择去相邻的景点(可能多个)中的一个或停留在原景点不动。而去这些地方所发生的概率是相等的。假设有 P 个景点与景点 M 相邻，它们分别是景点 R 、景点 S ，……景点 Q ，在时刻 T 可可处在景点 M ，则在 $(T+1)$ 时刻，可可有的可能在景点 R ，有的可能在景点 S ，……，有的可能在景点 Q ，还有可能停在景点 M 。

我们知道，聪聪是很聪明的，所以，当她在景点 C 时，她会选一个更靠近可可的景点，如果这样的景点有多个，她会选一个标号最小的景点。由于聪聪太想吃掉可快了，如果走完第一步以后仍然没吃到可可，她还可以在本段时间内再向可可走近一步。

在每个时间单位，假设聪聪先走，可可后走。在某一时刻，若聪聪和可可位于同一个景点，则可怜的可可就被吃掉了。

灰姑娘想知道，平均情况下，聪聪几步就可能吃到可可。而你需要帮助灰姑娘尽快的找到答案。

【输入文件】

从文件 *cchkk.in* 中读入数据。

数据的第 1 行为两个整数 N 和 E ，以空格分隔，分别表示森林中的景点数和连接相邻景点的路的条数。

第 2 行包含两个整数 C 和 M ，以空格分隔，分别表示初始时聪聪和可可所在的景点的编号。

接下来 E 行，每行两个整数，第 $i+2$ 行的两个整数 A_i 和 B_i 表示景点 A_i 和景点 B_i 之间有一条路。

所有的路都是无向的，即：如果能从 A 走到 B ，就可以从 B 走到 A 。

输入保证任何两个景点之间不会有多于一条路直接相连，且聪聪和可可之间必有路直接或间接的相连。

【输出文件】

输出到文件 *cchkk.out* 中。

输出 1 个实数，四舍五入保留三位小数，表示平均多少个时间单位后聪聪会把可可吃掉。

【输入输出样例】

【输入样例 1】

```
4 3
1 4
1 2
2 3
3 4
```

【输出样例 1】

```
1.500
```

【样例说明 1】

开始时，聪聪和可可分别在景点 1 和景点 4。

第一个时刻，聪聪先走，她向更靠近可可(景点 4)的景点走动，走到景点 2，然后走到景点 3；假定忽略走路所花时间。

可可后走，有两种可能：

第一种是走到景点 3，这样聪聪和可可到达同一个景点，可可被吃掉，步数

为 1，概率为 $\frac{1}{2}$ 。

第二种是停在景点 4，不被吃掉。概率为 $\frac{1}{2}$ 。

到第二个时刻，聪聪向更靠近可可(景点 4)的景点走动，只需要走一步即和可可在同一景点。因此这种情况下聪聪会在两步吃掉可可。

所以平均的步数是 $1 * \frac{1}{2} + 2 * \frac{1}{2} = 1.5$ 步。

【输入样例 2】

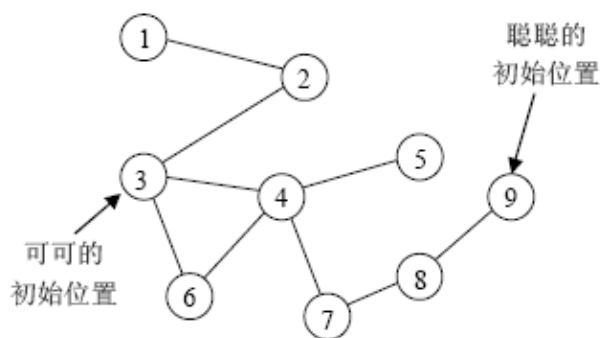
9 9
9 3
1 2
2 3
3 4
4 5
3 6
4 6
4 7
7 8
8 9

【输出样例 2】

2.167

【样例说明 2】

森林如下图所示：



【数据范围】

对于所有的数据， $1 \leq N, E \leq 1000$ 。

对于 50% 的数据， $1 \leq N \leq 50$ 。

【算法分析】

设 $dp[i,j,k]$ 表示时间 i ，猫在 j ，老鼠在 k 时，猫平均还需要多少步吃掉老鼠。

$dp[i,j,k] = \sum dp[i+1, x_j, k_u] / (en+1) + 1$ ，其中 x_j 是猫走两步（或一步）后的位置， k_u 是包括 k 在内所有与 k 相连的点， en 是 k 的出度。为了求猫的位置，预处理时作一次 ASSP（原来想的是 Dijkstra+Heap 优化，后来发现只要 BFS 就可以了）即可。但时间复杂度是 $O(n^3)$ 。注意到在推节点的过程中，如果在某一时间猫和老鼠分别在 tc, tm ，那么在以后的时间，猫和老鼠不会再在 tc, tm ，否则猫永远不会捉到老鼠。那么在设计状态的时候就没有必要考虑时间。设 $dp[i,j]$ 表示猫在 i ，老鼠在 j 时，猫平均还需要几步可以吃掉老鼠。于是有： $dp[i,j] = \sum dp[x_i, j_u] / (en+1) + 1$ 这样时间复杂度降到 $O(n^2)$

(From NOI 2005 Day2)

巧克力

有袋子里均匀地装着 c 种颜色的巧克力, 每种巧克力均有无限多。佳佳每次从袋子里拿一块放在桌子上, 如果桌子上已经有一块颜色相同的巧克力, 佳佳就把两块巧克力都吃掉。佳佳一共取出 n 块巧克力, 请问最后桌上有 m 块的概率有多大? 例如 $c=5$, $n=100$, $m=2$ 时, 概率为 0.625。

【分析】

由于 $m > c$ 或者 $m > n$ 或者 m 和 n 不同奇偶性时概率为 0, 以下解法中, 均认为 $m \leq \min\{c, n\}$, 且 m 和 n 同奇偶。用递推法解决。设 $d[i, j]$ 为取出 i 块巧克力, 恰好有 j 块的概率, 则

$$d[i, j] = d[i-1, j-1] \times (c-j+1)/c + d[i-1, j+1] \times (j+1)/c$$

算法的复杂度为 $O(nc)$ 。

这道题目是 2002 年国际大学生程序设计竞赛北京赛区比赛的题目。笔者当时负责评测此题, 发现几乎所有选手都是采用此法解题, 其中大部分选手超时, 而勉强通过测试的程序运行时间也很长。其实本题更好的做法是用生成函数。假设佳佳不在中途吃掉任何巧克力, 那么本题实际上是要让 m 种颜色取奇数个, $c-m$ 种颜色取偶数个, 求排列有多少种。

由刚才的结论, 奇数项指数生成函数为 $(e^x - e^{-x})/2$, 偶数项生成函数为 $(e^x + e^{-x})/2$ 。因此选 m 个奇数, $c-m$ 个偶数的指数生成函数为 $[(e^x - e^{-x})^m (e^x + e^{-x})^{c-m}] / 2^c$ 。当然, 由于是哪 m 种颜色取奇数个并不确定, 因此方案数还要乘以 $C(c, m)$, 再除以总方案数为 c^n 就得到了概率。

由于 $m \leq c$, 因此可以 $O(c^2)$ 的时间里把生成函数展开成 e^x 的有理式。展开以后可以依次考虑每一项中 x^n 的系数, 然后加起来就可以了。下面来说明样例的求解过程。

根据刚才的分析, 本题的答案为多项式 $p(x) = [(e^x - e^{-x})^m (e^x + e^{-x})^{c-m}] / 2^c \times c(c, m) / c^n$ 的 x^n 项系数乘以 $n!$ 。在样例中 $c=5, n=100, m=2$, 因此 $p(x) = [(e^x - e^{-x})^2 (e^x + e^{-x})^3] / 2^5 \times 10 / 5^{100}$ 。

暂时不管系数, 把多项式 $p'(y) = (y - y^{-1})^2 (y + y^{-1})^3$ 展开, 其中 $y = e^x$ 。由多项式乘法, $p'(y) = (y^2 - 2 + y^{-2})(y^3 + 3y + 3y^{-1} + y^{-3}) = (y^5 - 2y^3 + y) + (3y^3 - 6y + y^{-1}) + (3y - 6y^{-1} + 3y^{-3}) + (y^{-1} - 2y^{-3} + y^{-5}) = y^5 + y^3 - 2y - 4y^{-1} + y^{-3} + y^{-5}$ 。

如何求出 $p'(y)$ 中 x^n 项的系数呢? $y^5 = e^{5x}$, 它的展开式是 e^x 的展开式用 $5x$ 替换 x 得到的, 因此 x^{100} 的系数为原 e^x 的 x^{100} 的系数乘以 5^{100} , 即 $1/100! \times 5^{100}$ 。请注意, 最后还要乘以 5^{100} 和 $100!$, 因此全部都约去了! y^5 中 x^{100} 的系数是 $1/100! \times (-5)^{100}$, 也可以约去! 但 y^3 中 x^{100} 的系数为 $1/100! \times 3^{100}$, 只能约去 $100!$, 而保留 3^{100} , 因此有:

设 $y = e^x$ 的多项式 $p'(y) = (y - y^{-1})^m (y + y^{-1})^{c-m}$ 的 y^k 系数为 a_k , 则本题的最后答案为: $\sum\{(a_k + a_{k-1})(k/c)^n\} / 2^c \times c(c, m) \ (k > 0)$ 。在样例中, 由于 $n=100$ 很大, 因此 $(3/5)^{100}$ 可以忽略不计, 最后结果为 $2 / 2^5 \times c(5, 2) = 0.625$ 。由于 $c \leq 100$, 所以 2^c 可能会比较大, 所以最好在做多项式乘法的时候把 2^c 拿进去, 即

结论: 令 $g(y) = (0.5y - 0.5y^{-1})^m (0.5y + 0.5y^{-1})^{c-m}$, y^k 的系数为 a_k , 则本题的答案为

$$\sum\{(a_k + a_{k-1}) \times (k/c)^n\} / 2^c \times c(c, m) \quad (k > 0)$$

计算 $(k/c)^n$ 可以用对数, 时间复杂度为 $O(1)$, 计算整个式子的时间复杂度为 $O(c)$, 加上多项式乘法的复杂度 $O(c^2)$ (能否不多项式乘法而直接得到各个所需要的 a_k , 进而把时间复杂度降低到 $O(c)$ 呢? 这个问题留给读者思考), 总的时间复杂度为 $O(c^2)$ 。当 n 比较大的时候, $(k/c)^n$ 还可以忽略, 甚至不需要做完整的多项式乘法, 计算量几乎为 $O(1)$ (可以证明, $n \geq 5000$ 时误差不超过 10^{-9})。

(From 《算法艺术与信息学竞赛》 P259)

通过对这十道题目的分析，我们大概可以窥得概率类题目一二。有时候，所谓概率值是一个伪装，其本质还是我们所熟悉的模型，如终极情报网(agent)。有时候，概率的出现考验一些数学能力，如神奇口袋，又如要用到母函数的巧克力。而更多的情况下，这类问题与动态规划思想联系在一起。把概率值作为一个整体进行运算，很少会用 $\text{概率} = \text{发生次数} / \text{总次数}$ 的做法。百事世界杯之旅(PEPSI) 单人纸牌都是其中比较基本的题目，神经衰弱(shinkei) 三国围棋对抗赛就稍微复杂，并行期望值 迷宫统计 聪聪和可可(cchkk) 都非常经典且有难度，值得反复揣摩。概率类问题可以很好的包装问题，也便于评测，能够将多方面知识包装成一种适于出题的形式，所以在信息学竞赛中有相当高的出现频率。